

AegisEye — The Complete FYP Starter Book

From zero to a working accident-detection prototype — models, datasets, tools, and a step-by-step path for a team that's never done this before.

Muhammad Abu Bakar · Daniyal & Team

Supervisor: Sir Majid Hussain · University of Lahore — Compiled July 2026 (v2)

Contents

Part 1 — What AegisEye Actually Is	2
1.1 The Technical Explanation	2
AegisEye Core Features	2
Pipeline	3
Tech Stack	3
1.2 The Plain-English Explanation (No Jargon)	4
Part 2 — Tools, Setup & Team Workflow	4
2.1 What You Actually Need	4
2.2 Google Colab — Complete Beginner Walkthrough	4
2.3 Downloading Datasets/Models from Kaggle	5
2.4 Downloading Datasets/Models from Roboflow	5
2.5 Downloading Datasets from Mendeley Data	6
2.6 Downloading from Hugging Face	6
2.7 Team Collaboration & Git Workflow	6
2.8 Local Environment (Only for the Frontend, Not Training)	6
2.9 The Frontend — What to Actually Use	7
Option A — Figma → React (Recommended)	7
Option B — Pure Python Frontends (if no one wants to touch React)	7
Part 3 — Pretrained Models, Ranked for Your Use Case	8
3.1 Accident Detection (Accident vs. Normal Traffic)	8
#1 — YOLOv8 Accident Detection (Kaggle Notebook) [TOP PICK] PRIMARY MODEL	8
#2 — Roboflow Hosted Accident Model (deploy instantly via API)	8
#3 — CNN Accident Detection (Kaggle Notebook)	8
#4 — PyTorch CNN — Accident Detection from CCTV Footage (Kaggle Notebook)	9
3.2 Local Vehicle Detection (Rickshaw / Tuk-Tuk / Loader / 70cc Bikes)	9
#5 — UVH-26 Pretrained Weights (YOLO11-S/X, RT-DETR-S/X, DAMO-YOLO-T/L) [TOP PICK] BEST FOR LOCAL VEHICLES	9
#6 — Auto-Rickshaw Detector (Roboflow) (mAP@50 96.3%)	9
3.3 Severity Scoring & Temporal Models	10

#7 — Car-Crash-Detection — YOLOv3 + Optical Flow Tracking (GitHub)	10
#8 — DoTA / Traffic-Accident-Detection — CNN + LSTM (GitHub)	10
#9 — Accident Detection System — CNN Video Classifier (GitHub)	10
#10 — ACCIDENT Benchmark — Evaluation Suite, Not a Model	10
3.4 At-a-Glance Comparison	10
3.5 Can You Upgrade the Most Accurate Model with Your Data?	11
Part 4 — Local Vehicle Accident Clips: The Honest Gap	12
Reference crash-dynamics datasets for step 2	13
Part 5 — Frequently Asked Questions	13
Part 6 — Reference	15
Original datasets and code	15
Datasets added in v2	15

Part 1 — What AegisEye Actually Is

1.1 The Technical Explanation

AegisEye is a **real-time road-accident detection and alert system** that runs on video from existing CCTV/IP cameras. It watches a live or recorded video stream, classifies each moment as accident or normal traffic, then for a detected accident it estimates an **Impact Severity Score** (from vehicle speed, collision angle, vehicle count, pedestrian proximity), clips the **10 seconds before the crash** as a **Digital Blackbox** evidence file, and fires an **alert with location** toward emergency services — with an insurance-ready incident report.

The twist that makes this a real FYP contribution rather than a copy of existing research: identifying accidents involving **local vehicles** — rickshaws, tuk-tuks, loaders, and 70cc bikes — which almost no existing accident-detection system is trained to recognise, because none of the big public datasets were filmed in South Asia.

AegisEye Core Features

#	Feature	What It Does
F1	Real-Time Accident Detection	AI model watches CCTV feed 24/7 and flags collisions automatically — no operator needed
F2	Local Vehicle Recognition	Detects Pakistani/South-Asian vehicle types (rickshaw, tuk-tuk, loader, Honda CD-70, Qingqi)
F3	Impact Severity Score	Quantifies crash severity from speed, collision angle, vehicle count, pedestrian proximity
F4	Digital Blackbox	Auto-clips the 10-second pre-crash video, watermarks it, and stores as tamper-evident evidence
F5	Emergency Alert Dispatch	Sends SMS/call to ambulance/police with GPS location and severity level via Twilio API
F6	Insurance Report	Auto-populates an incident report with timestamp, location, severity, vehicle types, clip link

#	Feature	What It Does
F7	Web Dashboard	Professional monitoring interface with live feeds, alert history, and report downloads
F8	Multi-Camera Support	Dashboard can manage several CCTV streams simultaneously

Pipeline

```

CCTV / IP Camera Feed
  |
  v
[ Vehicle/Accident Detector ]  <-- pretrained model, fine-tuned by you
  |
  v
[ Severity Engine ]  -- speed (optical flow) + angle + vehicle count
  |
  v
[ Digital Blackbox ]  -- clips + watermarks pre-crash footage
  |
  v
[ Alert + Report Dispatch ]  -- ambulance/police alert + insurance report
  |
  v
[ Web Dashboard ]  -- what your team actually builds & demos

```

Tech Stack

Layer	Tool	Why
Vision / detection	YOLOv8 / YOLO11 (Ultralytics)	Fastest path to a working detector, huge community
Motion / severity	OpenCV + Farneback optical flow	Estimates vehicle speed and angle from frames, no extra hardware
Training environment	Google Colab / Kaggle Notebooks	Free GPU — your laptops don't do the heavy lifting
Backend / API	FastAPI or Flask	Serves the model, handles alert dispatch
Alerts	Twilio API	SMS/call dispatch to ambulance/police
Video clipping	FFmpeg	Auto-clips the pre-crash Digital Blackbox segment
Frontend	Figma → React or Reflex	Professional web UI — see Section 2.9
Version control	Git + GitHub	Team collaboration

1.2 The Plain-English Explanation (No Jargon)

Think of AegisEye as a very attentive security guard who never blinks, watches the traffic camera 24/7, and has one job: notice when a crash happens and immediately call for help.

- **The eyes** — an AI model that has looked at thousands of photos/videos of accidents so many times that it can recognise the visual pattern of a crash, the same way you instantly recognise a friend's face.
- **The brain** — once it sees something that looks like a crash, it does quick maths using the video frames (how fast were things moving, at what angle did they hit) to guess how bad the crash is.
- **The memory** — it automatically saves the 10 seconds of video right before the crash, like a car's blackbox recorder, so there's proof of what happened.
- **The phone call** — it then sends an automatic alert with the location to the ambulance/police, and later helps fill out the paperwork for insurance.
- **The local expert** — unlike existing systems, AegisEye knows what a rickshaw, a loader, a tuk-tuk, and a Honda CD-70 look like, because you teach it with South Asian vehicle data.

You are not building the eyes from scratch — that would take years and a research lab. Instead you take an eye that's already trained (a **pretrained model**) on general traffic accidents, and you show it a few thousand extra photos of rickshaws, tuk-tuks, and loaders so it learns to recognise those too. This is called **fine-tuning** or **transfer learning**. That's the whole trick, and it's exactly how real companies build these systems — nobody trains from zero anymore.

Part 2 — Tools, Setup & Team Workflow

2.1 What You Actually Need

Your laptops with weak integrated graphics are completely fine. **You will never train a model locally.** Here's the actual checklist:

- A Google account (for Colab + Drive)
- A Kaggle account (free)
- A Roboflow account (free tier)
- A Hugging Face account (free)
- A GitHub account plus one shared repository for the team
- Chrome or any browser — everything heavy runs in the cloud
- Python 3.10+ installed locally **only for running the lightweight frontend**, not for training

2.2 Google Colab — Complete Beginner Walkthrough

Colab is a Jupyter notebook that runs on Google's servers — including a free GPU. This is where all your model training happens.

1. Go to colab.research.google.com and sign in.
2. Click **New Notebook**. You'll see a page of "cells" — boxes for code or notes.
3. Turn on the free GPU: **Runtime** → **Change runtime type** → **Hardware accelerator** → **T4 GPU** → **Save**.
4. Click the play button on a cell to run it. Try `print("hello")` first.
5. To use files that survive between sessions, mount your Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

6. Colab's storage disappears when the session ends — **always save trained model weights to Google Drive**, not just to Colab disk.
7. Free sessions last up to ~12 hours and disconnect if idle — save progress every meaningful step.

Tip: Kaggle Notebooks work almost identically and currently give a more reliable 30 free GPU-hours per week with 9-hour sessions and background execution (training continues after you close the tab). Rotate between the two when one runs out.

2.3 Downloading Datasets/Models from Kaggle

1. On kaggle.com, go to your account settings → **Create New API Token**. This downloads kaggle.json.
2. In Colab, upload it and set it up:

```
from google.colab import files
files.upload() # choose kaggle.json when prompted
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle --quiet
```

3. Then download any dataset with one line (copy the exact command from the dataset's "Copy API command" button on its Kaggle page):

```
!kaggle datasets download -d asefjamilajwad/car-crash-dataset-ccd
!unzip -q car-crash-dataset-ccd.zip -d ccd_dataset
```

2.4 Downloading Datasets/Models from Roboflow

Roboflow gives you a private API key and a ready-made Python snippet on every dataset page (click **Download Dataset** → **show download code**):

```
!pip install roboflow --quiet
from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("traffic-u05r8").project("accident-detection-model-6rv64")
dataset = project.version(1).download("yolov8")
```

Get your key from roboflow.com → Settings → Roboflow API.

Never commit your API key to GitHub. Store it in a separate untracked file (e.g. .env).

2.5 Downloading Datasets from Mendeley Data

No API key needed — open the dataset page on data.mendeley.com, click **Download all files**, or pull the direct link straight into Colab:

```
!wget -O dataset.zip "<paste the Mendeley download link here>"
!unzip -q dataset.zip
```

2.6 Downloading from Hugging Face

Hugging Face hosts both models and datasets. Two options:

```
# Option A: datasets library
!pip install datasets --quiet
from datasets import load_dataset
ds = load_dataset("smart-dashcam/motorcycle-accident-driving-datasets")

# Option B: direct snapshot download
!pip install huggingface_hub --quiet
from huggingface_hub import snapshot_download
snapshot_download("smart-dashcam/motorcycle-accident-driving-datasets",
                  local_dir="./motorcycle_data")
```

2.7 Team Collaboration & Git Workflow

- **One shared GitHub repo** for all code. Each person works on their own branch (e.g. `feature/severity-engine`) and opens a Pull Request to merge into `main` — this avoids overwriting work.
- **Never push datasets or trained model weights to GitHub** — too large, GitHub will reject them. Add a `.gitignore` with `*.pt, *.zip, /datasets/`. Store those in a shared Google Drive folder and put the Drive link in the README.
- **Backups:** Colab autosaves notebooks to Google Drive if opened from Drive. Also enable **File** → **Save a copy in GitHub** from Colab's menu so notebook history lives in the repo too.
- **Communication:** WhatsApp/Discord group for quick chat, plus a free Trello or Notion board with columns To Do / In Progress / Done.
- **Suggested role split for 3–4 people:** (1) Data engineer — collects/cleans datasets, (2) Model engineer — trains/fine-tunes models, (3) Backend/integration — severity engine, alert dispatch, FFmpeg clipping, (4) Frontend — dashboard. Everyone reviews everyone's PRs.

2.8 Local Environment (Only for the Frontend, Not Training)

Use this only to run and test your web app locally — never for training. Your GPUs live in Colab/Kaggle.

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Keep a single `requirements.txt` in the repo so everyone's environment matches exactly.

2.9 The Frontend — What to Actually Use

You're right that plain Python doesn't have great built-in UI/UX. But there are two solid paths, and yes — **you can absolutely use Figma for UI/UX design and export it as real React code.**

Option A — Figma → React (Recommended)

Design in Figma (free for students), then use one of these plugins to export production-ready React code:

Tool	Best for	Link
Anima (~2M installs)	Most popular, generates React/HTML/Vue with responsive layouts	animaapp.com
Locofy	AI-powered, clean component-based React code	locofy.ai
Builder.io Visual Copilot	Highest production-quality React output	builder.io
Figma Make	Built into Figma, generates React directly	figma.com/make

Workflow: Design in Figma → export React with Anima/Locofy → connect the generated components to your FastAPI backend → deploy. Expect 20–40% manual cleanup on the generated code (responsive tweaks, data binding).

There are hundreds of **free dashboard UI kits** on the Figma Community — search "admin dashboard" or "monitoring dashboard", pick a dark-themed one, and customise it for AegisEye. This is how you get a genuinely professional final result without being a frontend expert.

Option B — Pure Python Frontends (if no one wants to touch React)

Option	Best for	Trade-off
Reflex (recommended)	Professional-looking web app, pure Python, compiles to React + FastAPI under the hood	Slightly more setup than Streamlit, much more polished result
Streamlit	Fastest way to a working demo	Looks noticeably "data-science-y"
Gradio	Quick one-off demo of the model alone	Not meant for full multi-page dashboards

Install and run Reflex:

```
pip install reflex
reflex init
reflex run
```

Recommendation: Go with **Figma → React** if any team member is even slightly comfortable with HTML/CSS/JS. Use **Reflex** if the entire team only knows Python.

Part 3 — Pretrained Models, Ranked for Your Use Case

Ranked by what matters most for a beginner team on a one-year timeline: real accuracy, how easy it is to get running in Colab this week, how well it matches CCTV (not dashcam) footage, and whether it helps with the local-vehicle angle. **Start at #1, add #5 and #6 once the core pipeline works.**

3.1 Accident Detection (Accident vs. Normal Traffic)

#1 — YOLOv8 Accident Detection (Kaggle Notebook) [TOP PICK] PRIMARY MODEL

Accuracy: ~91%. Trained on 3,200+ images over 25 epochs in Colab. Fork it, swap in the datasets below, and you have a working baseline in your first week. This is your primary model. To add AegisEye’s local-vehicle awareness: retrain with the merged dataset (Sorokh-Poth + Auto-RickshawImageBD + Roboflow accident sets), add vehicle classes to `data.yaml`, and fine-tune.

Pair with these datasets:

- CarCrashDataset (CCD) — 1,500 crash + 3,000 normal dashcam videos
- ACCIDENT (CVPR 2026 benchmark) — CCTV-specific, temporal + spatial labels, 2,027 real + 2,211 synthetic clips
- Road Accidents from CCTV Footages (Kaggle) — accident vs non-accident frames, some severity labels
- Roboflow Accident Detection Model dataset — 3,250 images, YOLOv8-ready
- **NTA Accident (Roboflow — NEW)** — 9,950 images, binary accident/non-accident, largest clean YOLO-ready set
- **YOLOproject Severity (Roboflow — NEW)** — 41,200 images, includes Mild/Moderate/Severe classes — directly maps to your severity scoring feature (F3)
- **HuggingFace Motorcycle Accident Videos (NEW)** — 68 accident + 68 driving videos, motorcycle-focused, use for video classification models (#8/#9) or extract frames for YOLO

#2 — Roboflow Hosted Accident Model (deploy instantly via API)

Already trained and hosted — call it with `InferenceHTTPClient`, zero training needed for a first working demo. Use this to get a live-looking prototype in days for your 7th-semester milestone, then swap in #1 once trained. **Limitation:** you cannot add custom classes to a hosted model, so for local vehicles run a separate detector (#5/#6) in parallel.

Pair with these datasets:

- Vehicle Crash Dataset (Roboflow) — 471 images, real-time alert focus
- Accident and Non-accident Image Dataset — clean binary classes, public domain
- **Accident detection (pfa-la3jd) — Roboflow (NEW)** — 3,140 images, good supplementary data

#3 — CNN Accident Detection (Kaggle Notebook)

Validation accuracy: ~83%. Simpler architecture than YOLOv8 — good to run first if the team wants to actually understand what’s happening inside the model before jumping to something heavier.

Pair with these datasets:

- Accident Detection from CCTV Footage — accident vs non-accident frames

- Car Accident Video Dataset (5,000 videos) — 3-sec clips before/during/after crash
- [saumilyajj/accident](#) (Roboflow — **NEW**) — 177 images with BOTH accident + auto-rickshaw + e-rickshaw + motorcycle labels; extremely rare combination, use as validation set

#4 — PyTorch CNN — Accident Detection from CCTV Footage (Kaggle Notebook)

PyTorch version rather than TensorFlow/Keras, built specifically on CCTV-style footage. Pick this if your team prefers PyTorch (which is also what UVH-26 and most modern research code uses, so it keeps your stack consistent).

Pair with these datasets:

- Accident Footages From CCTV (raw) — quick real-world sanity-check clips
- SAFE DRIVE DATASET — annotated, **shot in Pakistan** (highest local relevance)
- **Road Accident Detection** (Roboflow — **NEW**) — 1,310 images with vehicle type + accident classes combined (bicycle, bus, car, motorcycle, person, truck)

3.2 Local Vehicle Detection (Rickshaw / Tuk-Tuk / Loader / 70cc Bikes)

#5 — UVH-26 Pretrained Weights (YOLO11-S/X, RT-DETR-S/X, DAMO-YOLO-T/L) [TOP PICK] BEST FOR LOCAL VEHICLES

+31.5% mAP50:95 over COCO-trained baselines on South Asian traffic. Trained on 26,646 real CCTV images from Bengaluru with a dedicated 3-Wheeler (auto-rickshaw/tuk-tuk) class, plus bikes, vans, and LCVs — the single best starting point for your “local vehicle” contribution. Fine-tune it on the merged dataset below and it will directly support F2 (Local Vehicle Recognition).

Pair with these datasets:

- UVH-26 dataset — 1.8M boxes, 14 India-specific vehicle classes
- Poribohon-BD — rickshaw, CNG, easy-bike, wheelbarrow classes
- Rickshaw classification (8 classes) — includes cargo-transport-rickshaw (~ loader)
- **Sorokh-Poth** (Mendeley — **NEW**) — **9,809 images, 10 classes: Auto-rickshaw, Bike, Bus, Bicycle, Car, CNG, Leguna, Rickshaw, Truck, Van.** The most valuable single dataset for your local vehicle contribution — CNG matches Qingqi, Leguna is loader-adjacent, Rickshaw is directly usable
- **Auto-RickshawImageBD** (Mendeley — **NEW**) — 1,331 YOLO-annotated images, day + night, 7 Bangladeshi cities. Only dataset with dedicated night-time rickshaw annotations

#6 — Auto-Rickshaw Detector (Roboflow) (mAP@50 96.3%)

mAP@50: 96.3%, Precision: 88.9%, Recall: 92.3%. Small dataset (112 images) but very high accuracy on the auto-rickshaw class — a fast add-on once UVH-26 is working, to sharpen rickshaw detection specifically.

Pair with these datasets:

- Auto-Rickshaw Dataset (4.5k images) — larger volume for fine-tuning
- Auto Rickshaw Detection (1.6k images, pretrained) — ready-to-use pretrained model + API
- Detecting Unauthorized Vehicles (Bangladesh) — 1,730 images, YOLOv8, mAP50 ~ 83.4%

- **Auto-RickshawImageBD (Mendeley — NEW)** — merge with the 112-image Roboflow set for a combined ~1,450 rickshaw training corpus

3.3 Severity Scoring & Temporal Models

#7 — Car-Crash-Detection — YOLOv3 + Optical Flow Tracking (GitHub)

Combines detection with motion/tracking logic — the closest existing reference architecture to your Impact Severity Score module (speed via optical flow + collision angle). Use as a **code reference**, not a drop-in model. Port the optical flow module into your pipeline as post-processing after YOLOv8 detection.

Pair with these datasets:

- CADP (Car Accident Detection and Prediction) — 1,416 YouTube segments, CCTV-perspective
- DETRAC Dataset — vehicle detection & tracking benchmark
- D2-City Dataset — 10,000+ dashcam accident clips

#8 — DoTA / Traffic-Accident-Detection — CNN + LSTM (GitHub)

Temporal model — trains on full video sequences so it can flag an accident **as it unfolds**, not just after. More advanced; good stretch goal for 8th semester once the core pipeline is solid.

Pair with these datasets:

- CarCrashDataset (GitHub source) — full training code included
- ACCIDENT (CVPR 2026 benchmark) — temporal + spatial localization labels
- **HuggingFace Motorcycle Accident Videos (NEW)** — 136 videos at 30 fps, motorcycle-specific, feed directly into CNN+LSTM

#9 — Accident Detection System — CNN Video Classifier (GitHub)

CNN video classifier built specifically for CCTV-style footage rather than dashcams — alternative reference if #1 doesn't fit your data well.

Pair with these datasets:

- IITH Accident Dataset (CCTV) — real Indian CCTV accident videos
- Accident Detection from CCTV Footage — accident vs non-accident frames
- **HuggingFace Motorcycle Accident Videos (NEW)** — motorcycle-focused videos for local relevance

#10 — ACCIDENT Benchmark — Evaluation Suite, Not a Model

A full benchmark (CVPR 2026) with baseline approaches and a leaderboard. Use it at the end to **evaluate and score** whichever model your team finishes training — gives you a defensible number for your report and defence.

Pair with these datasets:

- ACCIDENT dataset — 2,027 real + 2,211 synthetic CCTV clips
- Mendeley — YOLOv8 + CNN-Transformer fusion paper — reference architecture + companion dataset

3.4 At-a-Glance Comparison

Model	Accuracy / mAP	Beginner	CCTV Angle	Local Vehicle Aware	Hosted API
YOLOv8 Acci- dent (#1)	~91% acc	Yes	Partial	After retraining	Yes
Roboflow Hosted (#2)	n/a	Yes	Partial	No	Yes
CNN Acci- dent (#3)	~83% val	Yes	Partial	After retraining	No
PyTorch CNN CCTV (#4)	n/a	Medium	Yes	After retraining	No
UVH- 26 (#5)	+31.5% mAP	Medium	Yes	Yes (built-in)	No
Auto- Ricksha (#6)	mAP@50 96.3%	Yes	Partial	Yes	Yes
Car- Crash- Detector (#7)	n/a	Medium	Yes	No	No
DoTA CNN+LS (#8)	n/a	Hard	Yes	No	No
Accident Sys (#9)	n/a	Medium	Yes	No	No
ACCIDE Bench- mark (#10)	n/a (eval only)	Medium	Yes	No	No

3.5 Can You Upgrade the Most Accurate Model with Your Data?

Yes, and this is exactly the recommended path. The accuracy upgrade sequence:

1. **Start with YOLOv8m (#1)** as your base — best balance of accuracy (~91%) and beginner friendliness.
2. **Upgrade to YOLO11m** (the latest Ultralytics model) for potentially higher accuracy — same training code, just change `YOLO('yolov8m.pt')` to `YOLO('yolo11m.pt')`.

3. **Fine-tune on your combined dataset** (Sorokh-Poth + Auto-RickshawImageBD + NTA Accident + YOLOproject Severity + SAFE DRIVE).
4. **Add internet accident clips** for validation: extract frames from YouTube clips using FFmpeg, annotate in Roboflow's free annotator, then use as a supplementary test set.
5. **Evaluate on the ACCIDENT Benchmark (#10)** for a defensible final number in your thesis.

Example training call:

```
from ultralytics import YOLO

model = YOLO('yolo11m.pt')          # or yolov8m.pt

model.train(
    data='aegiseye_combined.yaml',  # your merged dataset config
    epochs=100,
    imgsz=640,
    batch=16,                        # drop to 8 if CUDA OOM
    device=0,
    project='aegiseye',
    name='accident_v1'
)
```

Your aegiseye_combined.yaml:

```
path: /content/drive/MyDrive/aegiseye_data
train: train/images
val:   val/images
test:  test/images

names:
  0: accident
  1: non-accident
  2: rickshaw
  3: cng
  4: bike
  5: car
  6: bus
  7: truck
  8: loader
  9: pedestrian
```

Part 4 — Local Vehicle Accident Clips: The Honest Gap

Straight answer: **no dataset** — Kaggle, Roboflow, Mendeley, or HuggingFace — has **actual accident clips specifically featuring Pakistani rickshaws, tuk-tuks, loaders, or 70cc bikes**. This was checked directly and it's a real gap, not a search miss. The realistic path around it:

1. **Detect the vehicle separately from the crash.** Fine-tune UVH-26 (#5) with Sorokh-Poth + Auto-RickshawImageBD + the Roboflow rickshaw sets so your model reliably recognises

rickshaws, tuk-tuks, loaders, and 70cc bikes in CCTV frames.

2. **Borrow crash dynamics from similar Asian traffic.** DAD (42.6% motorcycle-car collisions), A3D, DADA-2000, and the HuggingFace Motorcycle Accident dataset are dashcam/video accident datasets from Asian traffic where motorbikes and tricycles are actually crashing — not labeled “Pakistani rickshaw” but the closest real crash dynamics available.
3. **Combine the two.** Run your vehicle detector (step 1) over the crash footage (step 2) to automatically flag “rickshaw/motorbike-involved accidents”, then manually verify a sample. This composite approach is a legitimate, documented technique when no labeled crash-participant dataset exists for a region.
4. **Collect a small local validation set yourselves.** Even 50–100 clips — from local news CCTV coverage of accidents, or your own recorded near-misses/staged clips — makes your final evaluation far more credible and is a defensible research contribution.

Reference crash-dynamics datasets for step 2

Dataset	Details	Link
DAD (Dashcam Accident)	678 users, 6 cities. 42.6% motorcycle-car, 15.6% motorcycle-motorcycle	arxiv/2505.07611
A3D	1,500 East-Asian dashcam clips, heterogeneous traffic incl. motorbikes/tricycles, temporally annotated	arxiv/1903.00618
DADA-2000	2,000 accident videos, Asian dashcam footage, participants include motorbike/cyclist	paperswithcode/dada-2000
HuggingFace Motorcycle	68 motorcycle accident + 68 driving videos, 30fps	huggingface.co/smart-dashcam

Part 5 — Frequently Asked Questions

Q: What is a “pretrained model” and why not build our own from scratch?

A pretrained model has already learned general patterns (edges, shapes, motion, what a vehicle looks like) from millions of images/videos. Fine-tuning it on your smaller, specific dataset (local vehicles, CCTV angles) takes hours on a free GPU instead of the months and huge datasets it’d take to train from zero. Every model in Part 3 works this way — this is standard industry practice, not a shortcut.

Q: What does “accuracy” vs “mAP” mean?

Accuracy is simply: out of all predictions, what % were correct — used for classification (accident vs no-accident). mAP (mean Average Precision) is used for object detection (finding and boxing a rickshaw in a frame) — it measures how well the predicted box overlaps the real object across many confidence thresholds. Higher is better for both; above ~85% is considered strong.

Q: Can I really use Figma for the frontend?

Yes. Design in Figma (free for students), export to React using Anima, Locofy, or Builder.io. See Section 2.9. This is genuinely how professional teams work in 2026.

Q: Can I upgrade the most accurate model with my own data and internet clips?

Yes. Start with YOLOv8m or YOLO11m, fine-tune on the combined dataset from Part 3.5. For YouTube clips: extract frames using `ffmpeg -i video.mp4 -vf fps=1 frame_%04d.jpg`, annotate in Roboflow’s free annotator, then use as supplementary training or validation data. YouTube’s terms restrict *redistributing* the video, not using it for local model training.

Q: My Colab session disconnected and I lost my training progress — now what?

This is why you save checkpoints. Add `model.save('/content/drive/MyDrive/checkpoints/epoch_10.pt')` every few epochs so you only ever lose a few minutes of work. Reconnect, reload the last checkpoint, continue.

Q: I get “CUDA out of memory” — what do I do?

Reduce batch size (try 8 or 4 instead of 16), reduce image resolution, or use a smaller model variant (YOLOv8n instead of YOLOv8x). The free T4 GPU has 16GB VRAM — plenty for these models at sensible batch sizes.

Q: Kaggle says “kaggle.json not found” — what’s wrong?

You either skipped uploading the file or it landed in the wrong folder. Re-run the upload cell in Section 2.3 and make sure the `!cp kaggle.json ~/.kaggle/` step completed without error.

Q: How do we actually test if the model is good enough?

Split every dataset into train / validation / test (70/15/15). Train only on train, tune on validation, and only report final numbers from the untouched test set. Build a confusion matrix (false alarms vs missed accidents). For a safety system, missed accidents matter more — tune your confidence threshold accordingly. Keep a human-verification step before any real alert is dispatched.

Q: What should the 30% code milestone (7th semester) actually contain?

A realistic, defensible 30%: (1) datasets collected & cleaned, (2) a baseline accident-detection model trained and evaluated (pick #1 or #2), (3) a first-pass local-vehicle detector (#5), (4) system architecture diagram + documentation, (5) an early frontend skeleton (even just a video feed and a detection overlay). You don’t need severity scoring, blackbox clipping, or real Twilio alerts working yet.

Q: How do we divide the work without stepping on each other?

Use the role split in Section 2.7 and Git branches per feature. Agree on the shared model output format early:

```
{'accident': True, 'confidence': 0.92, 'vehicles': ['rickshaw', 'car']}
```

so backend and frontend can build against that shape before the model is fully trained.

Q: What if our supervisor asks “has this been done before”?

Yes, accident detection exists in research. What’s genuinely yours: **severity quantification from fixed CCTV** (not dashcams), **automated digital blackbox + dispatch**, **insurance-ready output**, and the strongest point — **detecting accidents involving local vehicle types that no existing open dataset or system covers**.

Part 6 — Reference

- Claude chat used to research and compile this book — open shared chat

Original datasets and code

- D2-City Dataset
- CADP (Car Accident Detection and Prediction)
- DETRAC Dataset
- DoTA / Traffic-Accident-Detection (GitHub)
- CarCrashDataset (GitHub)
- Accident Detection System (GitHub)
- Car-Crash-Detection (GitHub)
- Roboflow YOLOv8 Accident Detection Model
- IITH Accident Dataset (CCTV)
- UVH-26 paper (arxiv)

Datasets added in v2

- HuggingFace Motorcycle Accident Dataset
- Auto-RickshawImageBD (Mendeley)
- Sorokh-Poth (Mendeley)
- Roboflow Accident Object Detection search
- Kaggle Accident Datasets search

Compiled July 2026 (v2) for AegisEye — Muhammad Abu Bakar, Daniyal & team. Supervisor: Sir Majid Hussain, University of Lahore. Verify each link still resolves and re-check license/usage terms before training or submission.